PROJECT REPORT

The Butterfly Effect

Interactive Event Planning Scenario Demonstrator

Required field	Value to be completed
Student name	[STUDENT ACTION REQUIRED: Enter your full name.]
GR number / Roll number	[STUDENT ACTION REQUIRED: Enter your GR number or roll number using the university's required field name.]
Division	[STUDENT ACTION REQUIRED: Enter your division.]
College / institute	[STUDENT ACTION REQUIRED: Enter the official college or institute name.]
University	[STUDENT ACTION REQUIRED: Enter the official university name if required.]
Faculty / guide	[STUDENT ACTION REQUIRED: Enter the faculty or guide name if required by the official format.]
Academic year / semester	[STUDENT ACTION REQUIRED: Enter the academic year and semester if required.]

STUDENT ACTION REQUIRED: Check the complete university format and page order. The supplied item was a small cover image only.

Prepared 29 October 2026

Submission format note

The report-format PDF referenced in the request was not present in D:\butterfly-effect. The supplied file is a small cover image and does not make chapter requirements, declarations, page order, or detailed formatting legible. This is a complete technical draft, not a claim of exact university-format compliance.

STUDENT ACTION REQUIRED: Obtain the full official format and adapt the report before submission.

Student details

Required field	Value to be completed
Student name	[STUDENT ACTION REQUIRED: Enter your full name.]
GR number / Roll number	[STUDENT ACTION REQUIRED: Enter your GR number or roll number using the university's required field name.]
Division	[STUDENT ACTION REQUIRED: Enter your division.]
College / institute	[STUDENT ACTION REQUIRED: Enter the official college or institute name.]
University	[STUDENT ACTION REQUIRED: Enter the official university name if required.]
Faculty / guide	[STUDENT ACTION REQUIRED: Enter the faculty or guide name if required by the official format.]
Academic year / semester	[STUDENT ACTION REQUIRED: Enter the academic year and semester if required.]

Contents

Abstract | 1. Introduction | 1.1 Problem statement | 1.2 Motivation and intended users | 2. Objectives | 3. Proposed solution | 4. Technology stack | 5. System design and architecture | 5.1 Workflow | 6. Implemented features | 7. AI tools and development method | 8. Testing methodology and results | 9. Results | 10. Limitations and future improvements | 11. Deployment readiness | 12. Conclusion | References | Testing results | Screenshot evidence

Abstract

The Butterfly Effect is a browser-based event-planning scenario demonstrator. It lets a user change six planning inputs and immediately see calculated spending, budget balance, preparation pressure, schedule risk, and a readiness category. A dependency map highlights pathways from changed inputs to modeled outcomes, and a fixed baseline remains available for comparison.

The application uses transparent deterministic rules implemented in TypeScript. It is intended for illustrating a simplified model, not for forecasting real events or proving scientific causality. The project was built with React and Vite and does not require a backend or an API key. Available verification records include a production build, lint run, six simulation tests, and browser interaction checks. The complete university report PDF was not present in the project; the supplied cover image does not expose its full section requirements.

1. Introduction

Planning an event involves connected decisions. Increasing the guest count changes catering expenditure; expenditure affects the remaining budget; and reducing preparation time raises the model's pressure and schedule-risk values. A person considering one decision at a time may miss these linked effects.

The Butterfly Effect provides a small interactive environment for exploring those relationships. The application focuses on event planning because costs, guest counts, available budget, and time remaining can be represented as clear inputs and outputs. The model is deliberately compact so its rules can be followed during a demonstration.

1.1 Problem statement

A static list of event costs does not make dependencies between planning choices and outcomes easy to explore. Users need a way to change assumptions, view recalculated results, and compare the changed scenario with a stable starting point.

1.2 Motivation and intended users

The project was prepared as an interactive university IPE demonstration. Its intended users are students, instructors, and other viewers who want to see how a small set of scenario assumptions affects a simple planning model. No user study or survey was conducted, so the report makes no claims about measured learning outcomes or user satisfaction.

2. Objectives

The project objectives are to provide editable event-planning variables; calculate related cost and schedule indicators deterministically; visualize modeled dependencies; compare a current scenario with a fixed baseline; provide quick presets and reset; and make the application usable in a browser without server-side services or API credentials.

3. Proposed solution

The application presents six controls: venue cost, guest count, catering cost per guest, travel cost, available budget, and days remaining. The simulation computes catering expenditure as guest count multiplied by catering cost per guest. Total expenditure is venue cost plus catering expenditure plus travel cost. Budget remaining equals available budget minus total expenditure; deficit is the positive overspend, or zero when spending stays within the budget.

Preparation pressure is derived from days remaining on a bounded 0–100 scale. Schedule risk is computed from that pressure. Readiness is assigned using the budget balance, schedule-risk value, and days remaining. The interface shows these relationships as an interactive graph and as result cards. Four presets alter one input at a time: double guests, increase venue cost, shorten preparation time, and increase budget.

4. Technology stack

The project manifest identifies React and React DOM 19.x, TypeScript 7.0.x, Vite 8.3.x, and Oxlint 1.89.x. The application is implemented in TSX and CSS. Node.js's built-in test runner executes the simulation tests. No additional test framework, backend, paid API, or runtime service is used. Versions above reflect the project manifest and lockfile in the inspected project.

5. System design and architecture

The interface layer is implemented in src/App.tsx. It owns the current scenario and selected graph node, renders the input controls and presets, calls the simulation functions, and displays results and comparison values. The calculation layer is src/simulation.ts, which contains scenario types, ranges, validation, and deterministic calculations. The test layer is tests/simulation.test.mjs. Vite serves the development application and produces static files in dist for deployment.

The baseline values are a separate fixed object in the simulation module. Editing a current value does not mutate the baseline. Reset copies the baseline into current scenario state. The graph uses an explicit list of modeled edges and highlights downstream nodes from changed or selected nodes.

5.1 Workflow

The user changes a slider or numeric field. The input is bounded to its allowed range and step. React state updates the current scenario; the deterministic simulation recomputes results; the dashboard and graph update; and the baseline comparison shows the changed input and output deltas. Selecting a graph node highlights its modeled downstream path.

Figure 2. Ripple map with an active change

STUDENT ACTION REQUIRED: Apply Double the guests and show the 300 guest value, recalculated spend, and highlighted graph path. Capture the actual rendered app and insert it in Section 5.1 after the workflow description.

6. Implemented features

• Scenario controls with six labeled range sliders and numeric inputs.

• Calculations for catering expenditure, total expenditure, budget remaining, budget deficit, preparation pressure, schedule risk, and readiness.

• An interactive SVG cause-and-effect graph with keyboard-operable node controls and downstream highlighting.

• A live current-versus-baseline comparison and per-input change indicators.

• Four presets and reset to the original default scenario.

• Responsive dark interface with visible keyboard focus indicators and no required backend or API key.

Figure 1. Application dashboard

STUDENT ACTION REQUIRED: Show the actual app at its default scenario with the scenario controls and Plan Outlook results visible. Open http://127.0.0.1:5173/ or run the local command in the handover section; reset to baseline; capture a screenshot; insert it in Section 6 after the feature list.

7. AI tools and development method

The initial project handoff states that the application had previously been developed using OpenCode. The original OpenCode conversation and prompt logs were not available in the files inspected for this report, so individual OpenCode prompts and contributions cannot be documented. The supplied Codex prompts and the project handover record show the later transition to Codex for diagnosis, repair, final verification, and report preparation.

Codex inspected the existing project rather than replacing it. The first recorded work repaired broken application state and rendering, made presets and reset functional, corrected graph dependencies, and produced a running preview. A later development pass added input validation, handled the zero-budget gauge case, clarified the illustrative nature of the model, removed the remote font request, and added six built-in Node tests. The final pass in this conversation is focused on preparing this report and prompt book; it did not modify the working application.

Prompt engineering evidence is provided separately in the prompt book. It reproduces the three available user prompts from the attached text files and summarizes outcomes supported by the project and recorded checks.

8. Testing methodology and results

The simulation tests use Node's built-in node:test and strict assertions. They cover baseline totals, exact budget deficit, deterministic pressure and risk values, range extremes and zero budget, rejection of invalid values, and variable ranges and updates. The records below distinguish automated test output from browser interaction checks performed in earlier project tasks.

Test	Expected	Observed	Status
Baseline totals and budget reconcile	INR 120,000 catering; INR 355,000 total; INR 145,000 remaining; zero deficit	Assertions passed for all four expected values	PASS — automated, previous final development pass
Budget deficit is exact overspend	INR 855,000 spend; INR 355,000 negative remaining; INR 355,000 deficit	Assertions passed for all expected values	PASS — automated, previous final development pass
Pressure, risk, determinism	At 7 days: 92% pressure and 66% risk; repeated calculation matches	Assertions passed; readiness expectation corrected to at-risk after first test run	PASS — automated, final run
Range extremes and zero budget	Finite outputs; INR 6,200,000 max-range spend; INR 6,150,000 deficit; zero budget handled	Assertions passed	PASS — automated, final run
Invalid values	NaN, infinity, and negative scenario input rejected	Expected RangeError assertions passed	PASS — automated, final run
Ranges and update helper	Guest range is 20–1,000 step 10; update preserves other fields	Assertions passed	PASS — automated, final run
Browser presets and reset	Presets update the intended input and derived values; reset restores baseline	Guest, venue, prep-time, budget presets and reset observed across prior and later browser checks	PASS — browser, prior task records
Production preview	Static production bundle serves the app and can interact	Rendered at localhost:4173; double-guests preset and reset observed	PASS — browser, prior final development pass
Console errors and graph keyboard selection	No runtime errors; keyboard selection works	No browser-console interface available; complete node-focus interaction not established	NOT VERIFIED

9. Results

For the default scenario, the model calculates catering expenditure of INR 120,000, total expenditure of INR 355,000, and INR 145,000 remaining from the INR 500,000 budget. It calculates 50% preparation pressure, 45% schedule risk, and "good" readiness. These values were observed in the rendered application during prior browser verification.

The browser record also shows the double-guests preset changing 150 guests to 300 and total expenditure to INR 475,000; the expensive-venue preset creating a INR 5,000 budget deficit; the prep-time preset changing 45 days to 15 days, 83% pressure, 62% risk, and "at-risk" readiness; the budget preset increasing budget to INR 600,000; and reset restoring the baseline. The later production-preview check confirmed initial default values and exercised double-guests and reset.

Figure 3. Baseline comparison

STUDENT ACTION REQUIRED: Show the current scenario beside the unchanged baseline and a visible change summary. Capture the actual rendered app after applying one preset and insert it in Section 9 after the results paragraph.

10. Limitations and future improvements

The model's rules are illustrative and intentionally simplified. It does not include vendor quotes, taxes, deposits, transportation routes, task dependencies, uncertainty, or probabilistic estimates. Its readiness categories are rule-based labels, not validated recommendations. The graph represents only dependencies encoded in the application.

Browser console inspection and a complete graph keyboard-selection test were not established in the available verification record. No user research, accessibility certification, performance benchmark, or public deployment was performed. The report format PDF was not available; the supplied image only partially shows a report cover.

Possible future work includes completing the university-format review, capturing genuine application screenshots, testing all controls and graph interactions across screen sizes, adding scenario save/export only if needed, and conducting a documented user evaluation if the course requires it.

11. Deployment readiness

This is a frontend-only Vite application. The production build output contains an HTML entry point and hashed CSS and JavaScript assets under dist. The current app has no application-level localhost URL dependency, server routes, or API credentials. A production preview was opened locally and rendered the dashboard. Deployment is not complete and no public URL exists.

A straightforward manual route is to run npm run build and upload the generated dist folder to Netlify Drop. Netlify's official instructions describe drag-and-drop publishing for a built output folder; its documentation says a manual deploy can produce a public URL without an account, while an account makes the deployment easier to claim and manage. Follow the current instructions at https://docs.netlify.com/start/quickstarts/netlify-drop-quickstart/ and https://docs.netlify.com/deploy/create-deploys/. Check the generated page and preset interactions at the resulting URL.

12. Conclusion

The Butterfly Effect is a functioning interactive demonstration of a deterministic event-planning model. It combines editable scenario inputs, calculated outputs, an explicit dependency graph, presets, reset, and a fixed-baseline comparison. Available records show successful build, lint, simulation tests, and browser checks. The main submission dependencies are the complete university format, student identity details, genuine screenshots if required, and final manual review.

References

Project source: src/App.tsx, src/simulation.ts, src/index.css, tests/simulation.test.mjs, package.json, and package-lock.json. Accessed during preparation of this report.

The Butterfly Effect project handover: PROJECT_HANDOVER.md. Used as a record of project scope and prior checks; checked against source files and the conversation's observed test output where available.

Netlify Docs, "Netlify Drop Quickstart" and "Create deploys." https://docs.netlify.com/start/quickstarts/netlify-drop-quickstart/ and https://docs.netlify.com/deploy/create-deploys/. Accessed 29 October 2026.

University report-format cover image supplied by the student: IPE_Project_report_Format.docx.png. The complete format PDF or DOCX was not supplied, so this image was insufficient to establish required sections or detailed formatting.

Appendix A. Declarations and acknowledgements

The supplied cover image does not establish whether the university requires a certificate, originality declaration, or acknowledgements page. Do not use invented institutional wording or signatures.

STUDENT ACTION REQUIRED: Insert the exact required pages and personally complete/sign fields where required.

Appendix B. Local project access

From PowerShell: `cd D:\butterfly-effect`, then `npm run dev`. Open http://127.0.0.1:5173/ while the dev server is running.